9
Pruebas de unidad



Nuestra profesión ha evolucionado mucho en los últimos 10 años. En 1997 nadie había oído hablar del Desarrollo guiado por pruebas (DGP). Para la mayoría, las pruebas de unidad eran pequeños fragmentos de código

desechable que creábamos para asegurarnos de que nuestros programas funcionaban. Escribíamos clases y métodos, y después código *ad hoc* para probarlos, lo que implicaba algún tipo de programa controlador que nos permitiera interactuar manualmente con el programa que habíamos escrito.

Recuerdo crear un programa de C++ para un sistema incrustado de tiempo real a mediados de la década de 1990. El programa era un sencillo temporizador con la siguiente firma:

```
void Timer::ScheduleCommand(Command* theCommand, int milliseconds)
```

La idea era sencilla; el método `execute` de `Command` se ejecutaba en un nuevo subproceso tras el número especificado de milisegundos. El problema era cómo probarlo. Confeccioné un sencillo programa controlador que escuchaba al teclado. Cada vez que se introducía un carácter, se programaba un comando que escribía el mismo carácter cinco segundos después. Introduje una rítmica melodía en el teclado y esperé a que se reprodujera en pantalla cinco segundos después:

> *«I… want-a-girl… just… like-the-girl-who-marr… ied… dear… old… dad.»*

Incluso tarareé la melodía mientras pulsaba la tecla, y la volví a cantar cuando aparecieron los puntos en la pantalla.

Ésa fue mi prueba. Cuando vi que funcionaba y se lo mostré a mis compañeros, me deshice del código de prueba.

Como he afirmado, nuestra profesión ha evolucionado mucho. Ahora crearía una prueba que garantizara el funcionamiento de hasta el más mínimo detalle del código. Aislaría el código del sistema operativo en lugar de invocar las funciones estándar de temporización. Las imitaría para tener control total sobre el tiempo. Programaría comandos que definieran indicadores Booleanos y avanzaría el tiempo, para observar los indicadores y asegurarme de que pasaran de `false` a `true` al cambiar el tiempo al valor correcto. Cuando superara una serie de pruebas, comprobaría que fueran adecuadas para todo el que tuviera que trabajar con el código. Me aseguraría de comprobar las pruebas y el código en el mismo paquete. Sí, hemos

avanzado mucho, pero nos queda mucho por avanzar. Los movimientos Agile y TDD han animado a muchos programadores a crear pruebas de unidad automatizadas y cada vez son más. Pero en esta alocada carrera por añadir pruebas a nuestra disciplina, muchos programadores han pasado por alto dos de los aspectos más sutiles e importantes de diseñar pruebas de calidad.

Las tres leyes del DGP

Todos sabemos que el DGP nos pide que primero creemos las pruebas de unidad, antes que el código de producción. Pero esa norma es sólo la punta del iceberg. Tenga en cuenta las tres siguientes leyes[37]:

- **Primera ley**: No debe crear código de producción hasta que haya creado una prueba de unidad que falle.
- **Segunda ley**: No debe crear más de una prueba de unidad que baste como fallida, y no compilar se considera un fallo.
- **Tercera ley**: No debe crear más código de producción que el necesario para superar la prueba de fallo actual.

Estas tres leyes generan un ciclo de unos 30 segundos de duración. Las pruebas y el código de producción se crean de forma conjunta, las pruebas unos segundos antes que el código. Si trabajamos de esta forma, crearemos decenas de pruebas al día, cientos al mes y miles al año. Si trabajamos de esta forma, las pruebas abarcarán todos los aspectos de nuestro código de producción. El tamaño de dichas pruebas, que puede ser similar al del código de producción, puede suponer un problema de administración.

Realizar pruebas limpias

Hace unos años me pidieron que dirigiera un equipo que había decidido explícitamente que su código de prueba no debía mantenerse con los mismos estándares de calidad que su código de producción. Podían incumplir las

reglas en sus pruebas de unidad. La premisa era «Rápido y directo». No era necesario que las variables tuvieran nombres adecuados, ni que las funciones de prueba fueran breves y descriptivas. No era necesario que el código de prueba estuviera bien diseñado. Bastaba con que funcionara y abarcara el código de producción.

Puede que algunos lectores empaticen con esta decisión. Puede que en el pasado creara el tipo de pruebas que cree para la clase `Timer`. Supone un gran paso crear ese tipo de pruebas desechables a diseñar una *suite* de pruebas de unidad automatizadas. Por ello, como el equipo que dirigía, puede decidir que pruebas incorrectas sea mejor que no tener pruebas.

Pero el equipo no se daba cuenta que tener pruebas incorrectas era igual o peor que no tener prueba alguna. El problema es que las pruebas deben cambiar de acuerdo a la evolución del código. Cuanto menos limpias sean, más difícil es cambiarlas. Cuando más enrevesado sea el código de prueba, más probabilidades de que dedique más tiempo a añadir nuevas pruebas a la *suite* que el empleado en crear el nuevo código de producción. Al modificar el código de producción, las pruebas antiguas comienzan a fallar y el desastre impide que las pruebas se superen, por lo que acaban por convertirse en un obstáculo interminable.

Entre versiones, aumentó el coste de mantener la *suite* de pruebas de mi equipo. Acabó por convertirse en la principal queja entre los desarrolladores. Cuando los directores preguntaron sobre este aumento, los desarrolladores culparon a las pruebas. Al final, se vieron obligados a descartar la *suite* de pruebas completa.

Pero sin una *suite* de pruebas perdieron la posibilidad de garantizar el funcionamiento esperado de los cambios en el código. Sin una *suite* de pruebas no podían asegurar que los cambios en una parte del sistema no afectaran a otras diferentes. Los defectos aumentaron, lo que propició que temieran realizar cambios. Dejaron de limpiar su código de producción por miedo a que los cambios fueran dañinos. El código de producción comenzó a corromperse. Al final, se quedaron sin pruebas, con un código de producción enmarañado y defectuoso, clientes frustrados y la sensación de que su esfuerzo les había fallado.

En cierto modo tenían razón. Su esfuerzo les había fallado. Pero fue su

decisión de permitir que las pruebas fueran incorrectas lo que provocó el fallo. Si hubieran empleado pruebas limpias, su esfuerzo no habría fallado. Puedo afirmarlo con cierta seguridad porque he participado y dirigido muchos equipos que han tenido éxito gracias a pruebas de unidad limpias.

La moraleja de la historia es sencilla: el código de prueba es tan importante como el de producción. No es un ciudadano de segunda. Requiere concentración, diseño y cuidado. Debe ser tan limpio como el código de producción.

Las pruebas propician posibilidades

Si sus pruebas no son limpias, las perderá. Y sin ellas pierde lo mismo que hace que su código de producción sea flexible. Sí, ha leído bien. Las pruebas de unidad son las que hacen que el código sea flexible y se pueda mantener y reutilizar. La razón es sencilla. Si tiene pruebas, no tendrá miedo a realizar cambios en el código. Sin pruebas, cada cambio es un posible error. Independientemente de la flexibilidad de su arquitectura, de la división del diseño, sin pruebas tendrá miedo a realizar cambios por la posibilidad de añadir errores no detectados.

Pero con las pruebas ese miedo desaparece. Cuanto mayor sea el alcance de sus pruebas, menos miedo tendrá. Podrá modificar el código con total impunidad, aunque su arquitectura no sea la mejor y el diseño sea mediocre. Podrá mejorar la arquitectura y el diseño sin miedo alguno.

Por tanto, disponer de una *suite* automatizada de pruebas de unidad que cubran el código de producción es la clave para mantener limpio el diseño y la arquitectura. Las pruebas proporcionan las posibilidades, ya que permiten el cambio.

Si sus pruebas no son limpias, la capacidad de modificar el código se verá limitada y perderá la posibilidad de mejorar la estructura de dicho código. Cuanto menos limpias sean las pruebas, menos lo será el código. En última instancia perderá las pruebas y el código se corromperá.

Pruebas limpias

¿Qué hace que una prueba sea limpia? Tres elementos: legibilidad, legibilidad y legibilidad. La legibilidad es sin duda más importante en las pruebas de unidad que en el código de producción. ¿Qué hace que una prueba sea legible? Lo mismo que en el código: claridad, simplicidad y densidad de expresión. En una prueba debe decir mucho con el menor número de expresiones posible.

Fíjese en el código de FitNesse del Listado 9-1. Estas tres pruebas son difíciles de entender y sin duda se pueden mejorar. Por un lado, hay mucho código duplicado [G5] en las invocaciones repetidas a `addPage` y `assertSubString`. Sobre todo, este código se carga con detalles que interfieren con la expresividad de la prueba.

Listado 9-1
SerializedPageResponderTest.java.

```
public void testGetPageHieratchyAsXml() throws Exception
{
   crawler.addPage(root, PathParser.parse(“PageOne”));
   crawler.addPage(root, PathParser.parse(“PageOne.ChildOne”));
   crawler.addPage(root, PathParser.parse(“PageTwo”));

   request.setResource(“root”);
   request.addInput(“type”, “pages”);
   Responder responder = new SerializedPageResponder();
   SimpleResponse response =
     (SimpleResponse) responder.makeResponse(
       new FitNesseContext(root), request);
   String xml = response.getContent();

   assertEquals(“text/xml”, response.getContentType());
   assertSubString(“<name>PageOne</name>”, xml);
   assertSubString(“<name>PageTwo</name>”, xml);
   assertSubString(“<name>ChildOne</name>”, xml);
}

public void testGetPageHieratchyAsXmlDoesntContainSymbolicLinks()
throws Exception
{
   WikiPage pageOne = crawler.addPage(root, PathParser.parse(“PageOne”));
   crawler.addPage(root, PathParser.parse(“PageOne.ChildOne”));
   crawler.addPage(root, PathParser.parse(“PageTwo”));

   PageData data = pageOne.getData();
   WikiPageProperties properties = data.getProperties();
```

```
    WikiPageProperty symLinks = properties.set(SymbolicPage.PROPERTY_NAME);
    symLinks.set("SymPage", "PageTwo");
    pageOne.commit(data);

    request.setResource("root");
    request.addInput("type", "pages");
    Responder responder = new SerializedPageResponder();
    SimpleResponse response =
      (SimpleResponse) responder.makeResponse(
        new FitNesseContext(root), request);
    String xml = response.getContent();

    assertEquals("text/xml", response.getContentType());
    assertSubString("<name>PageOne</name>", xml);
    assertSubString("<name>PageTwo</name>", xml);
    assertSubString("<name>ChildOne</name>", xml);
    assertNotSubString("SymPage", xml);
  }

  public void testGetDataAsHtml() throws Exception
  {
    crawler.addPage(root, PathParser.parse("TestPageOne"), "test page");

    request.setResource("TestPageOne");
    request.addInput("type", "data");
    Responder responder = new SerializedPageResponder();
    SimpleResponse response =
      (SimpleResponse) responder.makeResponse(
        new FitNesseContext(root), request);
    String xml = response.getContent();

    assertEquals("text/xml", response.getContentType());
    assertSubString("test page", xml);
    assertSubString("<Test", xml);
  }
```

Fíjese en las invocaciones de `PathParser`. Transforman cadenas en instancias de `PagePath` usadas por las arañas. Esta transformación es totalmente irrelevante para la prueba y únicamente complica su cometido. Los detalles circundantes a la creación del respondedor y la obtención y conversión de la respuesta también sobran. También la forma de crear la URL de solicitud a partir de un recurso y un argumento (contribuí a crear este código, por lo que tengo todo el derecho a criticarlo).

Al final, el código no se ha diseñado de forma legible. El lector se ve rodeado de miles de detalles que debe comprender antes de que las pruebas tengan sentido.

Fíjese ahora en las pruebas mejoradas del Listado 9-2. Hacen exactamente lo mismo, pero se han refactorizado de forma más clara y

descriptiva.

Listado 9-2
SerializedPageResponderTest.java (refactorizado)

```
public void testGetPageHierarchyAsXml() throws Exception {
   makePages(“PageOne”, “PageOne.ChildOne”, “PageTwo”);

   submitRequest(“root”, “type:pages”);

   assertResponseIsXML();
   assertResponseContains(
     “<name>PageOne</name>”, “<name>PageTwo</name>”, “<name>ChildOne</name>”
   );
}

public void testSymbolicLinksAreNotInXmlPageHierarchy() throws Exception {
   WikiPage page = makePage(“PageOne”);
   makePages(“PageOne.ChildOne”, “PageTwo”);

   addLinkTo(page, “PageTwo”, “SymPage”);

   submitRequest(“root”, “type:pages”);

   assertResponseIsXML();
   assertResponseContains(
     “<name>PageOne</name>”, “<name>PageTwo</name>”, “<name>ChildOne</name>”
   );
   assertResponseDoesNotContain(“SymPage”);
}

public void testGetDataAsXml() throws Exception {
   makePageWithContent(“TestPageOne”, “test page”);

   submitRequest(“TestPageOne”, “type:data”);

   assertResponseIsXML();
   assertResponseContains(“test page”, “<Test”);
}
```

El patrón Generar-Operar-Comprobar[38] es evidente en la estructura de las pruebas. Cada una se divide claramente en tres partes. La primera crea los datos de prueba, la segunda opera en dichos datos y la tercera comprueba que la operación devuelva los resultados esperados.

Comprobará que se ha eliminado gran parte de los detalles molestos. Las pruebas son concisas y sólo usan los tipos de datos y funciones que realmente necesitan. Todo el que lea estas pruebas sabrá rápidamente para qué sirven y no se perderá entre detalles irrelevantes.

Lenguaje de pruebas específico del dominio

Las pruebas del Listado 9-2 ilustran la creación de un lenguaje específico del dominio para sus pruebas. En lugar de usar las API que los programadores emplean para manipular el sistema, creamos una serie de funciones y utilidades que usan dichas API y que facilitan la escritura y la lectura de las pruebas. Estas funciones y utilidades se convierten en una API especializada usada por las pruebas. Son un lenguaje de pruebas que los programadores usan personalmente para crear sus pruebas y para ayudar a los que después las lean.

Esta API de pruebas no se diseña con antelación, sino que evoluciona con la refactorización continuada del código de prueba. Al igual que refactorizamos el Listado 9-1 en el Listado 9-2, los programadores disciplinados refactorizan su código de prueba en versiones más sucintas y expresivas.

Un estándar dual

En un sentido, el equipo que mencionamos antes tenía razón. El código de la API de pruebas tiene un conjunto de estándares de ingeniería diferentes al código de producción. También tiene que ser sencillo, sucinto y expresivo, pero no tan eficaz como el código de producción. Después de todo, se ejecuta en un entorno de prueba, no de producción, y cada entorno tiene sus propias necesidades.

Fíjese en la prueba del Listado 9-3. La creé como parte de un prototipo de sistema de control medioambiental. Sin entrar en detalles, se aprecia que esta prueba comprueba que la alarma de baja temperatura, el calentador y el fuelle estén activados cuando la temperatura sea demasiado fría.

Listado 9-3
EnvironmentControllerTest.java

```
@Test
   public void turnOnLoTempAlarmAtThreashold() throws Exception {
     hw.setTemp(WAY_TOO_COLD);
```

```
      controller.tic();
      assertTrue(hw.heaterState());
      assertTrue(hw.blowerState());
      assertFalse(hw.coolerState());
      assertFalse(hw.hiTempAlarm());
      assertTrue(hw.loTempAlarm());
    }
```

Aquí hay muchos detalles. Por ejemplo, ¿para qué sirve la función `tic`? De hecho, la ignoraría mientras leemos esta prueba. Intente centrarse en saber si está de acuerdo en que el estado final del sistema tiene que ver con que la temperatura sea demasiado baja.

Al leer la prueba, la vista tiene que cambiar entre el nombre del estado comprobado y el sentido del estado comprobado. Vemos `heaterState` y después la vista salta a `assertTrue`. Vemos `coolerState` y nos fijamos en `assertFalse`. Resulta tedioso y dificulta la lectura de la prueba.

He conseguido mejorar la legibilidad de la prueba transformándola en el Listado 9-4.

Listado 9-4
EnvironmentControllerTest.java (refactorizado)

```
  @Test
    public void turnOnLoTempAlarmAtThreshold() throws Exception {
      wayTooCold();
      assertEquals("HBchL", hw.getState());
    }
```

Evidentemente, he ocultado el detalle de la función `tic` creando una función `wayTooCold`. Pero lo importante es la extraña cadena de `assertEquals`. Las mayúsculas significan activado y las minúsculas desactivado, y las letras siempre aparece en este orden: `{heater, blower, cooler, hi-temp-alarm, lo-temp-alarm}`.

Aunque prácticamente sea un incumplimiento de las reglas de asignación mental[39], en este caso parece apropiado. Una vez que conocemos el significado, la vista pasa por la cadena y podemos interpretar los resultados. La lectura de la prueba es casi un placer. Fíjese en el Listado 9-5 y compruebe con qué facilidad entiende las pruebas.

Listado 9-5
EnvironmentControllerTest.java (una selección mayor).

```
@Test
   public void turnOnCoolerAndBlowerIfTooHot() throws Exception {
     tooHot();
     assertEquals(“hBChl”, hw.getState());
   }

   @Test
   public void turnOnHeaterAndBlowerIfTooCold() throws Exception {
     tooCold();
     assertEquals(“HBchl”, hw.getState());
   }

   @Test
   public void turnOnHiTempAlarmAtThreshold() throws Exception {
     wayTooHot();
     assertEquals(“hBCHL”, hw.getState());
   }

   @Test
   public void turnOnLoTempAlarmAtThreshold() throws Exception {
     wayTooCold();
     assertEquals(“HBchL”, hw.getState());
   }
```

La función `getState` se reproduce en el Listado 9-6. No es un código muy eficaz. Para que lo sea, deberíamos haber usado `StringBuffer`.

Listado 9-6
MockControlHardware.java.

```
public String getState() {
   String state = “”;
   state += heater ? “H” : “h”;
   state += blower ? “B” : “b”;
   state += cooler ? “C” : “c”;
   state += hiTempAlarm ? “H” : “h”;
   state += loTempAlarm ? “L” : “l”;
   return state;
}
```

`StringBuffer` es poco atractivo. Incluso en código de producción, intento evitarlo si el coste es mínimo, como podría suceder en el Listado 9-6. Pero esta aplicación es claramente un sistema incrustado en tiempo real y es probable que los recursos del equipo y la memoria estén limitados. Sin

embargo, el entorno de pruebas es improbable que lo esté. Es la naturaleza del estándar dual. Hay cosas que nunca haría en un entorno de producción totalmente válidas para un entorno de prueba. Suelen ser problemas de memoria o eficacia de la CPU, pero nunca problemas de limpieza.

Una afirmación por prueba

Existe una escuela de pensamiento[40] que afirma que todas las funciones de prueba de una prueba JUnit sólo deben tener una instrucción de afirmación. Puede parecer una regla draconiana pero la ventaja se aprecia en el Listado 9-5. Las pruebas llegan a una misma conclusión, que se entiende de forma rápida y sencilla.

¿Pero qué sucede con el Listado 9-2? No parece razonable afirmar que el resultado es XML y que contiene determinadas subcadenas. Sin embargo, podemos dividir la prueba en dos, cada una con una afirmación concreta, como se muestra en el Listado 9-7.

Listado 9-7
SerializedPageResponderTest.java (una sola afirmación).

```
public void testGetPageHierarchyAsXml() throws Exception {
   givenPages(“PageOne”, “PageOne.ChildOne”, “PageTwo”);

   whenRequestIsIssued(“root”, “type:pages”);

   thenResponseShouldBeXML();
}

public void testGetPageHierarchyHasRightTags() throws Exception {
   givenPages(“PageOne”, “PageOne.ChildOne”, “PageTwo”);

   whenRequestIsIssued(“root”, “type:pages”);

   thenResponseShouldContain(
     “<name>PageOne</name>”, “<name>PageTwo</name>”, “<name>ChildOne</name>”
   );
}
```

He cambiado los nombres de las funciones para usar la convención *dado-cuando-entonces*[41]. De este modo las pruebas son más fáciles de leer. Desafortunadamente, al dividir las pruebas se genera código duplicado.

Podemos eliminar los duplicados por medio del patrón Método de plantilla[42] e incluir las partes *dado/cuando* en la clase base, y las partes entonces en derivaciones diferentes. O podríamos crear una clase de prueba independiente e incluir las partes *dado* y *cuando* en la función `@Before` y las partes entonces en cada función `@Test`. Pero parece un mecanismo excesivo para un problema tan menor. Al final, opto por las afirmaciones múltiples del Listado 9-2. Considero que la regla de una sola afirmación es una directriz adecuada[43]. Siempre intento crear un lenguaje de pruebas específico del dominio que la complemente, como en el Listado 9-5, pero no rechazo incluir más de una afirmación en una prueba. Creo que lo mejor que podemos decir es que el número de afirmaciones de una prueba debe ser mínimo.

Un solo concepto por prueba

Puede que una regla más indicada sea probar un único concepto en cada función de prueba. No queremos extensas funciones que prueben una cosa diferente tras otra, como sucede en el Listado 9-8. Esta prueba debería dividirse en tres diferentes que probaran tres cosas distintas. Al combinarlas en la misma función se obliga al lector a determinar por qué cada sección se ubica en ese punto y qué prueba dicha sección.

Listado 9-8

```
/**
* Varias pruebas para el método addMonths().
*/
public void testAddMonths() {
   SerialDate d1 = SerialDate.createInstance(31, 5, 2004);

   SerialDate d2 = SerialDate.addMonths(1, d1);
   assertEquals(30, d2.getDayOfMonth());
   assertEquals(6, d2.getMonth());
   assertEquals(2004, d2.getYYYY());

   SerialDate d3 = SerialDate.addMonths(2, d1);
   assertEquals(31, d3.getDayOfMonth());
   assertEquals(7, d3.getMonth());
   assertEquals(2004, d3.getYYYY());
```

```
    SerialDate d4 = SerialDate.addMonths(1, SerialDate.addMonths(1, d1));
    assertEquals(30, d4.getDayOfMonth());
    assertEquals(7, d4.getMonth());
    assertEquals(2004, d4.getYYYY());
  }
```

Las tres funciones deberían ser las siguientes:

- Dado el último día de un mes con 31 días (como mayo):
  1. Cuando se añade un mes, si el último día de ese mes es el 30 (como en junio), entonces la fecha debe ser el día 30 de ese mes, no el 31.
  2. Cuando se añaden dos meses a esa fecha, si el último mes tiene 31 días, entonces la fecha debe ser el día 31.
- Dado el último día de un mes con 30 días (como junio):
  1. Cuando se añade, si el último día de ese mes tiene 31 días, entonces la fecha debe ser el 30, no el 31.

Expresado de esta forma, se aprecia que existe una regla general entre las distintas pruebas. Al incrementar el mes, la fecha no puede ser mayor que su último día. Esto implica que al incrementar el mes en el 28 de febrero debe generarse el 28 de marzo. Falta esa prueba y convendría que la escribiéramos.

Así pues, no son las múltiples afirmaciones del Listado 9-8 las causantes del problema, sino el hecho de que se prueba más de un concepto. Probablemente la regla óptima sea minimizar el número de activos por concepto y probar un solo concepto por función de prueba.

F.I.R.S.T.[44]

Las pruebas limpias siguen otras cinco reglas, cuyas iniciales forman las siglas FIRST en inglés:

Rapidez (*Fast*): Las reglas deben ser rápidas y ejecutarse de forma rápida. Si lo hacen lentamente, no las ejecutará con frecuencia. Al no hacerlo, no detectará los problemas con la suficiente antelación como para solucionarlos. No se sentirá con libertad para limpiar el código, que acabará corrompiéndose.

Independencia (*Independent*): Las pruebas no deben depender entre ellas. Una prueba no debe establecer condiciones para la siguiente. Debe poder ejecutar cada prueba de forma independiente y en el orden que desee. Si las pruebas dependen unas de otras, la primera que falle provocará una sucesión de fallos, dificultará el diagnóstico y ocultará efectos posteriores.

Repetición (*Repeatable*): Las pruebas deben poder repetirse en cualquier entorno. Debe poder ejecutarlas en el entorno de producción, en el de calidad y en su portátil de camino a casa en un tren sin red. Si no puede repetir las pruebas en cualquier entorno, siempre tendrá una excusa de su fallo. También verá que no puede ejecutar las pruebas si el entorno no está disponible.

Validación automática (*Self-Validating*): Las pruebas deben tener un resultado booleano: o aciertan o fallan. No debe tener que leer un extenso archivo de registro para saber si una prueba ha acertado, ni comparar manualmente dos archivos de texto distintos para ello. Si las pruebas no se validan automáticamente, el fallo puede ser subjetivo y la ejecución de las pruebas puede requerir una extensa evaluación manual.

Puntualidad (*Timely*): Las pruebas deben crearse en el momento preciso: antes del código de producción que hace que acierten. Si crea las pruebas después del código de producción, puede que resulte difícil probarlo. Puede decidir qué parte del código de producción sea demasiado difícil de probar. No diseñe código de producción que no se pueda probar.

Conclusión

Apenas hemos abordado la superficie de este tema. De hecho, se podría crear un libro entero sobre pruebas limpias. Las pruebas son tan importantes para la salud de un proyecto como el código de producción. Puede que incluso más, ya que conservan y mejoran la flexibilidad, capacidad de mantenimiento y reutilización del código de producción. Por ello, intente que sean limpias. Trabaje para que resulten expresivas y concisas. Invente API de prueba que actúen como lenguaje específico del dominio que le ayude a crear las pruebas.

Si deja que las pruebas se corrompan, sucederá lo mismo con el código de

producción. Mantenga limpias las pruebas.

Bibliografía

- **[RSpec]**: *RSpec: Behavior Driven Development for Ruby Programmers*, Aslak Hellesay, David Chelimsky, Pragmatic Bookshelf, 2008.
- **[GOF]**: *Design Patterns: Elements of Reusable Object Oriented Software*, Gamma et al., Addison-Wesley, 1996.